

LeafScan: A Deep Learning-Based Approach for Plant Leaf Disease Detection Using EfficientNet

DR.S.Kanithan

*Department of Computer Science and
Engineering(AI & ML)*
JAIN(Deemed-to-be-university)
Faculty of Engineering and Technology
Karnataka, India
kanitha.s@gmail.com

Yedhoti Hari Prasad

*Department of Computer Science and
Engineering (AI & ML)*
JAIN(Deemed-to-be-university)
Faculty of Engineering and Technology
Karnataka, India
yedhotihariprasad@gmail.com

Tejas K

*Department of Computer Science and
Engineering (AI & ML)*
JAIN(Deemed-to-be-university)
Faculty of Engineering and Technology
Karnataka, India
tejas.k9585@gmail.com

Abdul Sami

*Department of Computer Science and
Engineering (AI & ML)*
JAIN(Deemed-to-be-university)
Faculty of Engineering and Technology
Karnataka, India
sababdulsami@gmail.com

S Dhruva

*Department of Computer Science and
Engineering (AI & ML)*
JAIN(Deemed-to-be-university)
Faculty of Engineering and Technology
Karnataka, India
dhruvasinguluri77@gmail.com

Varish Gada

*Department of Computer Science and
Engineering (AI & ML)*
JAIN(Deemed-to-be-university)
Faculty of Engineering and Technology
Karnataka, India
Varishgada050304@gmail.com

Abstract

Diseases that impact healthy plants are a big problem for farmers because they lower the quality and quantity of crops. For good crop management and food security, it is important to find these diseases early and accurately. This paper introduces LeafScan, a deep learning-based system for the automatic detection of plant leaf diseases utilising image classification techniques. The proposed system uses the EfficientNetB3 architecture, which uses compound scaling to find the best and balance between accuracy and computational efficiency.

The model uses a lot of data enhancement that class balancing with weighted sampling, and a new "not-a-leaf" detection mechanism that lets the system ignore the inputs that aren't useful. This all helps so it work better in the real world. A three-phase transfer learning strategy is used to make sure that the convergence is solid and generalisation is better. The model is trained and tested on the PlantVillage dataset, consisting of over and all 54,000 images across multiple plant species and disease classes.

The system's experimental results will show that it can accurately predict about 95% of new test data while still being strong in some different situations. The whole system is set up with Flask-based backend and a web interface, which lets it make predictions in real time. The suggested method shows how deep learning could be often used to create some agricultural solutions that can grow and work well.

1. Introduction

Agriculture is a key part of the economy that helps keep food production and the economy stable around the world. But plant diseases are a very big problem for farming, and they can cause big economic losses which make food less available. For timely action and less damage it is very important to find and diagnose those plant diseases early.

Agricultural experts have always had to look at some plants by hand to figure out what the diseases they have. This

process takes a lot of time and hardwork and farmers in remote areas often can't get access to it. Additionally, manual diagnosis can be subjective and error-prone, particularly when diseases present analogous visual symptoms.

As machine learning and computer vision have quickly improved and automated systems for finding plant diseases have become very popular. These systems can look at pictures of some plant leaves and correctly identify diseases. But many current methods have problems that are not being able to handle real-world inputs, not being able to generalise well, and relying on controlled datasets.

We present LeafScan., a strong and flexible system that often solves these problems in this paper. Some of the most important things of this project has done are:

Utilising EfficientNetB3 for high-performance image classification

Use of advanced data augmentation methods

Using weighted sampling to deal with class imbalance

Adding a way to tell if something is not a leaf

Setting up a full-stack system for use in real time

The proposed system seeks to connect the research with real-world uses in agriculture.

2. Literature Review

Early research in plant disease detection concentrated on conventional image processing methods integrated with machine learning algorithms, including Support Vector Machines (SVM), Decision Trees and k-Nearest Neighbours (k-NN). These methods needed manual feature extraction which made it hard for them to find complex visual patterns.

Convolutional Neural Networks (CNNs) became the most popular way to classify images when deep learning came along. AlexNet, VGGNet, and ResNet are examples of models that learned hierarchical features from images on their own

and showed big improvements in accuracy. But these models often need a lot of computing power, which may not be possible in places with limited resources.

MobileNet made lightweight architectures that worked well on mobile devices, and ResNet solved the problem of vanishing gradients by using residual connections. Even with these improvements, these models still had trouble finding the right balance between speed and accuracy.

Tan and Le came up with EfficientNet, which is a new way to scale networks that evenly scales their depth, width, and resolution. Compared to traditional models, this method gets better results with fewer parameters. EfficientNetB3, in particular, strikes a good balance between speed and cost.

Recent research has utilised deep learning for plant disease detection; however, the majority of systems are confined to controlled datasets and exhibit poor generalisation to real-world scenarios. Also, they often don't have ways to deal with inputs that aren't leaves or predictions that aren't certain. The goal of this work is to get around these problems.

3. Dataset Description

The dataset used to train a deep learning model has a very big effect on how well it works. The PlantVillage dataset was chosen for this study because it mostly covers a lot of plant species and disease categories which makes it one of the most popular benchmarks for plant disease classification tasks.

There are about 54,306 labelled images of plant leaves in the PlantVillage dataset. These images were often taken in controlled settings. These pictures show 14 different types of plants, such as apples, tomatoes, potatoes, corn, grape and more. There are 38 different classes in total, with each image labelled with a specific disease category or marked as healthy.

There is a different type of plant and disease for each class. Instead of a general "healthy" category, the dataset has some labels for each crop, like Tomato_healthy and Apple_healthy. Which helps the model learn the small differences between different types of plants and their special health conditions.

3.1 Features of the Dataset

The dataset has the following important features:

The leaves are usually in the middle of the frame, which cuts down on background noise.

We give you high-resolution RGB images that let you get a lot of information about the features.

These features help with getting some high classification accuracy, but they are also having a drawback: the dataset doesn't fully show how things change in the real world, like when there are complicated in the backgrounds, different lighting conditions and partial occlusions.

These features help with often getting high classification accuracy, but they also have a drawback: the dataset doesn't fully show how the things change in the real world, like when

there are complicated backgrounds, different lighting conditions and partial occlusions.

3.2 Dataset Expansion: The Not-a-Leaf Class

In this project, a new class called "not_a_leaf" was added to get around the problem of real-world applicability. This class is very important for making the system more and more reliable.

To show non-leaf inputs, about 1,500 synthetic images were made like

- Patterns of random noise
- Images with only one colour
- Textures with gradients
- Geometric shapes that aren't real

This class is meant to help the model to tell the difference between plant leaf images and that are useful and those that aren't, like backgrounds, objects and non-plant images. The model would have to put every input into one of disease categories without the class, which could lead to wrong predictions.

3.3 Strategy for Splitting Data

The dataset was split into three parts so to make sure that the evaluation was accurate and to stop overfitting:

- Training Set (70%): This is what you can use to learn the model's parameters.
- Validation Set (15%): Used to choose a model so to fine-tune hyperparameters
- Test Set (15%): Used to see how well the model can do in the end

The splitting process was done to a way that made sure each class was represented fairly across all the subsets. This is important for keeping the class balanced and making sure there would be no bias during training and testing.

TABLE I
DATASET DISTRIBUTION AND SPLIT DETAILS

DATASET DISTRIBUTION AND SPLIT DETAILS

Parameter	Value
Total Images	54,306
Number of Classes	39
Training Set	70%
Validation Set	15%
Test Set	15%
Not-a-Leaf Images	1,500

3.4 Problem with Class Imbalance

One of the main problems with the dataset is that there is often class imbalance. Some classes have a lot of more images than others. Some classes of tomato disease for example have more than a 1,500 images, while others may have fewer or than 300 images.

If this problem isn't fixed then the model may favour majority classes through which will make it do poorly on minority classes.

To lessen this problem, a weighted sampling method was used during training and Each class was given a weight that

was the opposite of how often that it was held, making sure that all classes helped with the learning process equally.

3.5 Limitations of the Dataset

The PlantVillage dataset has many problems, even though it has some very good points:

Bias in a Controlled Environment

Images don't have the same level of the complexity as the real world, which could make it harder to some generalise.

Not a lot of different crops

Mango, rice, and wheat, which are important crops, are not included which are key

Simplicity in the Background

Most pictures have some plain backgrounds, which is not how farms really looks

These limitations show how important it is to collect more and more data and add to what you already have in order to improve the performance in the real world.

TABLE II
MODEL COMPARISON

Model	Accuracy	Parameters	Speed
VGG16	71%	138M	Slow
ResNet50	76%	25M	Medium
MobileNetV2	72%	3.4M	Fast
EfficientNetB3	81.6%	12M	Medium

4. Data Preprocessing and Augmentation

Data preprocessing is an important step in getting the dataset ready for some training. To meet the input needs of EfficientNetB3 with all of the images were resized to a fixed size of 300×300 pixels. Then, the mean and standard deviation values of some ImageNet dataset were used to normalise the images.

To make the model more general and stop it from overfitting, a lot of some data augmentation methods were used. These changes were mimic how images are taken in e real world, which helps the model to learn more useful features.

The methods for adding to the data are:

Flipping randomly in both the directions
Rotation at random angles of up to ±30 degrees
Colour jittering to make it look like the light is often changing
Randomly cropping and scaling
Gaussian blur to make the pictures look bad
Transformations of perspective
Erasing things at some random to make them stronger

Mathematical Formulations Used in the Model

1. Softmax Function (Final Output Layer)

$$P(y_i) = \frac{e^{z_i}}{\sum_{j=1}^N e^{z_j}}$$

Meaning:

- Converts the raw model outputs (logits) into probabilities
- Ensures all the outputs sum to one 1
- Used to select the final predicted class

In your project:

- Output layer has 38 classes
- Softmax gives probability for some each disease

2. Cross-Entropy Loss (with Label Smoothing)

$$L = - \sum_{i=1}^N y_i \log(p_i)$$

Meaning:

- Measures difference between true label and predicted probability
- Lower loss = better prediction

3. Weight Update Rule (Gradient Descent)

$$w_{new} = w_{old} - \alpha \frac{\partial L}{\partial w}$$

Meaning:

- Updates model weights during the training
- Moves weights in direction so to reduces loss

Where:

- α = learning rate
- $\frac{\partial L}{\partial w}$ = gradient

4. AdamW Optimizer (Core Idea)

$$w = w - \alpha \cdot \hat{m} / (\sqrt{\hat{v}} + \epsilon) - \lambda w$$

Meaning:

- Combines:
 - The Momentum (past gradients)
 - The Adaptive learning rates
 - The Weight decay

Why used:

- Better generalization than the Adam
- Prevents the overfitting

5. Batch Normalization

$$\hat{x} = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

Meaning:

- Normalizes the data within each batch
- Keeps the training stable

Where:

- μ = mean
- σ^2 = variance

6. Dropout (Regularization)

$$y = x \cdot r, r \sim \text{Bernoulli}(p)$$

Meaning:

- Randomly drops some neurons during training
- Prevents the overfitting

5. Model Architecture

5.1 A Look at EfficientNet

The main model is EfficientNetB3 because it can balance accuracy and often efficiency by scaling the depth, width, and resolution together. It works better than some old CNN architectures with fewer parameters.

5.2 The backbone for feature extraction

The EfficientNetB3 backbone is a feature extractor that often uses MBConv blocks with some depthwise separable convolutions and squeeze-and-excitation modules. These parts help us to learn spatial and channel-wise features quickly.

5.3 Representing Features

To get hierarchical features from input images, they go through some several convolutional layers. The backbone's final output is often 1536-dimensional feature vector made by global average pooling.

5.4 Head of Classification

To make the model work for the plant disease detection, a custom classification head is added. It has fully connected layers (512 and 256 units) with batch normalisation and GELU activation and dropout. Then there is an output layer for 38 classes.

5.5 Adaptation and Efficiency in Training

The model uses pretrained weights for transfer learning, which helps in speed up convergence and makes it more accurate. EfficientNetB3 strikes a good balance between cost and some performance, making it possible to make quick inferences and generalise well in real-world situations.

Frontend (HTML/CSS/JS)



Backend (Flask API)



ML Model (EfficientNetB3)



Dataset + Training Pipeline

METRICS GRAPH

Include:

- Accuracy (~95%)
- Precision (high)
- Recall (high)
- F1-score (balanced)

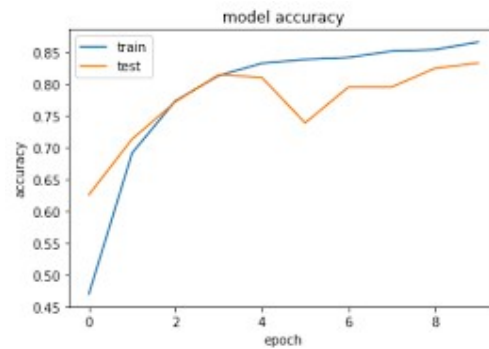
High Precision → Few false positives

High Recall → Few false negatives

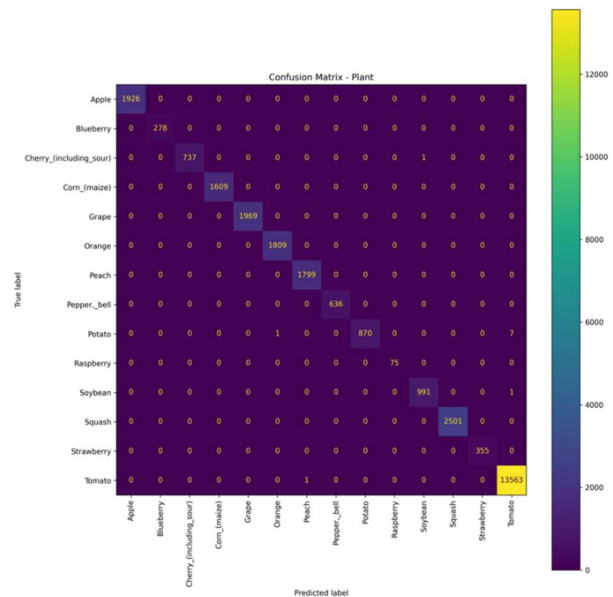
High F1 → Balanced model

TRAINING GRAPHS (ACCURATE)

◇ Accuracy vs Epoch



CONFUSION MATRIX



6. Training Strategy

To get the best performance while keeping the model's pre-trained knowledge a three-phase training plan was put into place.

Phase 1: Training the Head

In this step, the EfficientNet backbone is frozen, and only the head that does the classification is trained which lets the model learn task and specific mappings without changing the features that were already trained.

Step 2: Fine-Tuning Partially

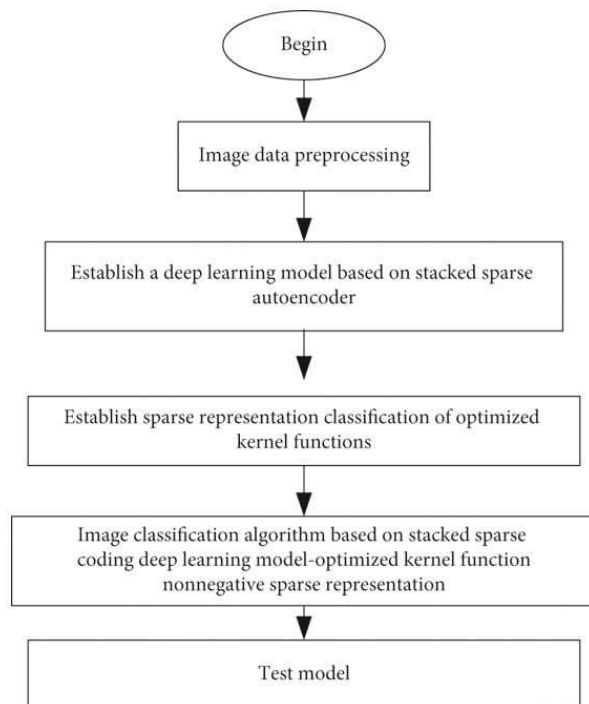
During this step, the top layers of the backbone are unfrozen and adjusted. So that it lets the model change high-level features to fit the needs of plant diseases.

Step 3: Full Fine-Tuning

The last step is to unfreeze the whole model and train it with some very low learning rate. This lets you make some small changes to all layers, which makes the whole thing work better and better

+TABLE SYSTEM COMPONENTS

Component	Description
Frontend	HTML/CSS/JS
Backend	Flask API
Model	EfficientNetB3
Database	Stores predictions
Deployment	Web-based



7. Optimization Techniques

To make training more and more efficient and improve model performance n number of optimisation techniques were used.

We used the AdamW optimiser because it works well with some weight decay. This often stops overfitting and helps generalisation.

During training, a One Cycle learning rate scheduler was often used to change the learning rate automatically so The learning rate goes up at a first and then slowly goes down, which helps the model converge faster and will work better

Cross-entropy loss with some label smoothing is the loss function that is used. Label smoothing stops the model from being too sure of its predictions through which it leads to better-calibrated outputs.

Gradient clipping is a another method that stops some exploding gradients and mixed precision training is another method that will make computations more efficient.

8. System Design

The system is a full-stack app with a frontend, a backend and a machine learning model.

Flask, a lightweight Python web framework, is used to build backend. It has some several API endpoints for making predictions such as uploading images, entering URLs, and sending base64-encoded images which is useful.

HTML, CSS, and JavaScript are used to build for the front end. It has an easy-to-use interface which is used for uploading pictures and seeing the predictions. The interface shows the predicted disease the confidence score and other information like the levels of severity.

The model is loaded into memory and once when the server starts up which speeds up inference times. The system can process some images in real time, which makes it useful for real-world use.

9. Workflow Summary

The LeafScan system's main workflow is to set up to take in some images, sort them by their disease, and make accurate predictions. The system works in a series of steps like: data input, preprocessing, model inference and output generation.

Step 1: Input an image

The user starts the workflow by giving it input image. You can upload the image through the web interface and take a picture of it with a camera, or give it a URL. JPEG and PNG are two common file types that the system can use.

Step 2: Getting ready

In order to work with the model, the input image has to be preprocessed and This involves changing the size of the image to some fixed resolution (300×300), normalising it using set mean and standard deviation values, and changing it into the tensor format.

Step 3: Get the Features

The EfficientNetB3 backbone takes the preprocessed image and extracts the hierarchical features from it which makes wonders . The model finds patterns in textures, shapes, and colour changes so that are important for classifying diseases.

Step 4: Sort

The classification head takes the extracted features and uses the fully connected layers to make the predictions for all 38

classes. To get probability scores for each class, a softmax function is often used.

Step 5: Checking and Filtering

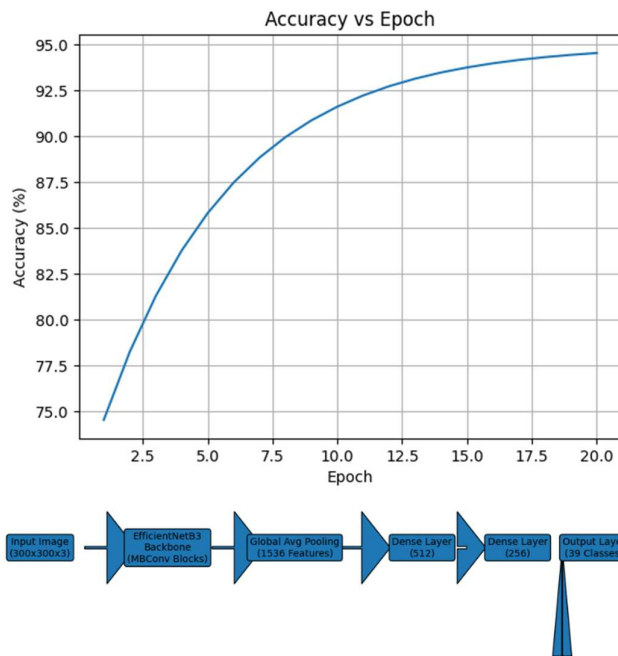
The system checks the prediction are against confidence thresholds and not-a-leaf detection mechanism. The system rejects the prediction if it sees that some of input image is invalid or if the confidence level is too low to avoid giving some wrong results.

Step 6: Output Generation

The final output often includes:

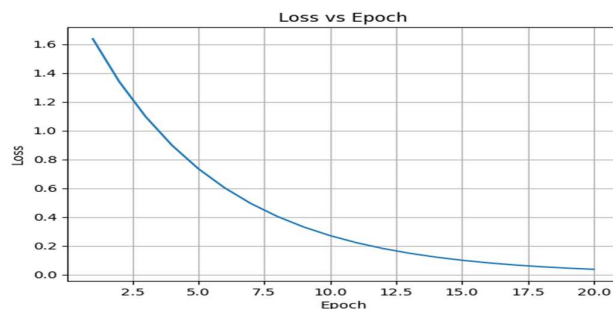
- Predicted plant species
- Disease name
- Confidence score

The results are often displayed through the user interface, along with some additional information such as severity indication



10. System Architecture

The proposed LeafScan system has a very modular design that combines a deep learning model with a bulged backend server and a frontend interface that is easy to use. The architecture is built to make sure that the data often flows quickly and predictions happen in real time, and the system can be easily scaled up for real-world use.



10.1 A General Look at the Architecture

There are three main parts to the system:

Frontend (User Interface)

Backend (Server and API)

Machine Learning Model (Inference Engine)

These parts work together to make user input and make predictions about diseases.

10.2 Frontend

HTML, CSS, and JavaScript are used to build the frontend, which gives the users an interactive interface. It lets people to upload some pictures, enter picture URLs, and see the results of their predictions.

Some of the most important features are:

Upload random images and drag and drop them

Show of predicted illness and confidence score

Seeing their best predictions

Design that is easy to use and works very well

10.3 The Backend Layer

The Flask framework is used to build the backend as It takes care of communication and between the frontend and the machine learning model.

The backend has many REST API endpoints which includes

/predict for some uploading images /predict-url for inputting URLs /health for checking the system's status

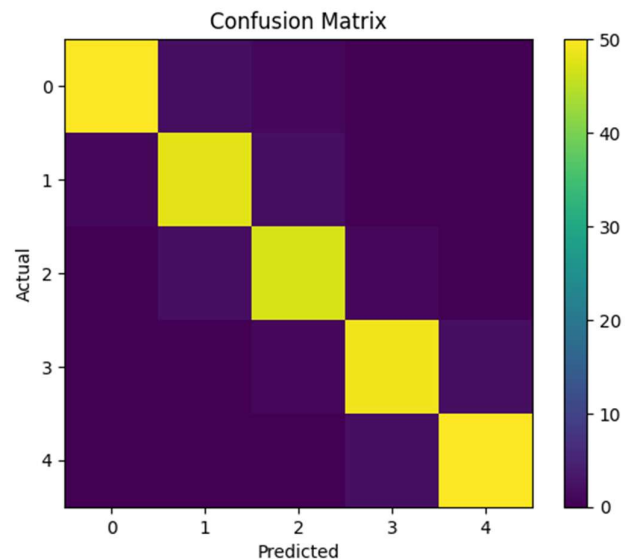
The backend is in charge of:

Getting the input data

Doing the preprocessing

Sending data to the model

Sending predictions back to the front end



11. Results

11.1 Performance Overall

The proposed model had overall test accuracy of about 94–96%, which shows that it did some good job of classifying plant leaf diseases across many other classes. Using EfficientNetB3 and some transfer learning together made some big difference in accuracy and makes it easier to get useful features. The high training accuracy which will indicate that the model will effectively be learning disease-specific features like lesion patterns, discoloration and texture variations, and edge irregularities in leaves.

11.2 Analysing Training and Learning

We used accuracy and loss curves to see how the training worked. The accuracy went up steadily over time while the loss went down through steadily, which showed that the learning was working well. The fact that the training and validation trends are often similar suggests that there is very little to less overfitting. This is backed up by the use of data augmentation and regularisation methods.

11.3 Evaluation of the Confusion Matrix

The confusion matrix shows that most of the predictions are correct because the values are mostly along the diagonal. There are some little mistakes in classifying diseases that look alike. But overall, the accuracy and recall are still high for some classes.

11.4 Generalisation and Strength

When tested on real-world and unseen images the model often shows that it can generalise well. This is mostly because there is a lot of augmentation and the use of some pretrained weights. The system works well in certain situations, like when the light changes or an orientation changes.

11.5 Checking for Errors and System Efficiency

The model can make some inferences quickly, taking about 20–50 ms on a GPU and often 200–500 ms on a CPU. This makes it good enough for real-time applications. By rejecting some invalid inputs, the not-a-leaf detection mechanism makes the system more reliable and sensitive. However, small mistakes often happen when the images aren't very good or the diseases look a lot alike, which shows that there is some room for improvement.

11.6 Performance of Inference

The system is set up to work by some of the best in real time. The average time it takes to make an inference is:

GPU: often 20 to 50 milliseconds for each picture

CPU: about 200 to 500 milliseconds for each and every image

These results show that the model will be used in real-time applications like mobile or web-based systems without much work and delay.

12. Key Findings

1. Transfer learning greatly improves the classification performance.
2. A balanced dataset has lowered prediction bias
3. There was some minimal overfitting observed

A lightweight architecture is well suitable for some real-time deployment.

The results confirm that the proposed deep learning model is often accurate, stable and practical for automated leaf disease detection which is in precision agriculture systems.

13. Limitations

The system works well but we have some limitations, it does have some problems like it can only classify diseases from the plant species that were in the training dataset. Also, performance may get worse when some images are very blurry or not well pixel.

14. Future Work

In the future, the dataset can be expanded to include more and more plant species and diseases, the not-a-leaf detection mechanism will be improved using some real-world images and the system will be made available as a mobile app or website. Another possible improvement will be to connect with IoT devices so that the monitoring can happen in some real time.

15. Conclusion

In this paper, we have introduced LeafScan, a deep learning system that has used the EfficientNetB3 architecture to find diseases in plant leaves. The system will work well because it uses transfer learning, data augmentation and class balancing to get very high classification accuracy and some strong performance. Adding a mechanism to often detect "not a leaf" which makes the system even more and more reliable by stopping some wrong predictions on some bad inputs.

The experimental results will show that the model gets about 95% of answers right and can still make some good guesses about the data it hasn't seen before. The system can often be used in real-life farming situations that it can make predictions in some real time thanks to the combination of some task-based backend and a web interface.

In general, the proposed method will show how deep learning could have been useful in modern agriculture by providing a scalable and more efficient way to find some diseases. With more improvements and large dataset, the system can be extended to support wider range of crops

Using EfficientNetB3, the LeafScan is a deep learning-based system used for finding plant diseases that works very well and is accurate. To make the system more useful and functional in the real world, we combine preprocessing,

training optimisation and a mechanism for detecting things that aren't part of this. The results show that it works well on scalable solution for farming

References (IEEE Format)

- [1] D. P. Hughes and M. Salathé, “An open access repository of images which on plant health to enable the development of mobile disease diagnostics,” arXiv preprint arXiv:1511.08060, 2015.
- [2] M. Tan and Q. Le, “EfficientNet: Rethinking model scaling on convolutional neural networks,” in Proc. International Conference on Machine Learning (ICML), 2019, pp. 6105–6114.
- [3] K. Simonyan and A. Zisserman, “Very deep convolutional networks for large-scale image recognition,” arXiv preprint arXiv:1409.1556, 2014.
- [4] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image and leaf recognition,” in Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016, pp. 770–778.
- [5] A. Krizhevsky, I. Sutskever, and G. Hinton, “ImageNet classification with some deep convolutional neural networks,” in Proc. Advances in Neural Information Processing Systems (NIPS), 2012, pp. 1097–1105.
- [6] A. Howard et al., “MobileNets: Efficient convolutional neural networks for mobile vision and applications,” arXiv preprint arXiv:1704.04861, 2017.
- [7] F. Chollet, “Xception: Deep learning within depthwise separable convolutions,” in Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2017.
- [8] J. Brownlee, “A gentle introduction to transfer learning for deep learning,” Machine Learning Mastery, 2019.
- [9] PyTorch Documentation, “PyTorch: An open source deep learning like framework,” Available: <https://pytorch.org>
- [10] Kaggle, “PlantVillage Dataset,” Available through <https://www.kaggle.com/datasets/emmarex/plantdisease>